

Deploying ICSC Connect on Ubuntu (EC2) + Apache2

Complete, copy-pasteable step-by-step guide. Adjust paths and the domain to your own.

Assumed: a fresh Ubuntu 24.04 / 26.04 EC2 instance, ports 22, 80, 443 open in the security group, and a domain (e.g. `school.example.com`) pointing to the EC2 public IP.

1. SSH in and update

```
ssh -i your-key.pem ubuntu@<your-ec2-ip>
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential curl git unzip software-properties-common
```

2. Install MongoDB 7

```
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | \
  sudo gpg -o /usr/share/keyrings/mongodb-server-7.0.gpg --dearmor
echo "deb [signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg] \
  https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" \
  | sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
sudo apt update
sudo apt install -y mongodb-org
sudo systemctl enable --now mongod
mongosh --eval 'db.runCommand({ping:1})' # should print { ok: 1 }
```

3. Install Python 3.11 + Node 20 + Yarn

```
sudo add-apt-repository -y ppa:deadsnakes/ppa
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3.11-dev

curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
sudo npm install -g yarn
node -v && yarn -v
```

4. Get the code on the server

Either upload via `scp` / `rsync` or `git clone` :

```
sudo mkdir -p /opt/icsc && sudo chown ubuntu:ubuntu /opt/icsc
cd /opt/icsc
# Option A - rsync from your laptop
# rsync -avz /app/ ubuntu@<ec2-ip>:/opt/icsc/
# Option B - git clone
# git clone <your-repo-url> .
```

You should end up with `/opt/icsc/backend` and `/opt/icsc/frontend` .

5. Backend setup

```
cd /opt/icsc/backend
python3.11 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Create the production `.env` :

```
cat > /opt/icsc/backend/.env <<'EOF'
MONGO_URL="mongodb://localhost:27017"
DB_NAME="icsc_school_db"
CORS_ORIGINS="https://school.example.com"
FRONTEND_URL="https://school.example.com"
JWT_SECRET="<paste 64+ random hex chars; openssl rand -hex 32>"
ADMIN_EMAIL="admin@yourschool.com"
ADMIN_PASSWORD="<strong password>"
SENDGRID_API_KEY=""
SENDGRID_FROM_EMAIL="no-reply@yourschool.com"
EMERGENT_LLM_KEY=""
APP_NAME="icsc-school"
EOF
chmod 600 /opt/icsc/backend/.env
```

Photo upload & daily-digest will work without `EMERGENT_LLM_KEY` only if you skip photo uploads; otherwise plug in any S3-compatible storage of your choice.

systemd service for the API

```
sudo tee /etc/systemd/system/icsc-backend.service > /dev/null <<'EOF'
[Unit]
```

```

Description=ICSC Connect FastAPI
After=network.target mongod.service

[Service]
User=ubuntu
WorkingDirectory=/opt/icsc/backend
Environment="PATH=/opt/icsc/backend/.venv/bin"
ExecStart=/opt/icsc/backend/.venv/bin/uvicorn server:app --host 127.0.0.1 --port 8001 --workers 2
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now icsc-backend
sudo systemctl status icsc-backend
curl -s http://127.0.0.1:8001/api/health    # {"status":"ok"}

```

6. Frontend production build

```

cd /opt/icsc/frontend
cat > .env <<'EOF'
REACT_APP_BACKEND_URL=https://school.example.com
ENABLE_HEALTH_CHECK=false
EOF
yarn install
yarn build          # output goes into /opt/icsc/frontend/build

```

7. Apache2 install + reverse proxy

```

sudo apt install -y apache2
sudo a2enmod proxy proxy_http headers rewrite ssl

```

Virtual host config:

```

sudo tee /etc/apache2/sites-available/icsc.conf > /dev/null <<'EOF'
<VirtualHost *:80>
    ServerName school.example.com

    DocumentRoot /opt/icsc/frontend/build
    <Directory /opt/icsc/frontend/build>
        Options -Indexes +FollowSymLinks
        AllowOverride None
        Require all granted

    # SPA fallback – all non-file routes go to index.html

```

```

RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteCond %{REQUEST_URI} !^/api/
RewriteRule ^ index.html [L]
</Directory>

# Reverse-proxy /api to FastAPI on 8001
ProxyPreserveHost On
ProxyPass /api/ http://127.0.0.1:8001/api/
ProxyPassReverse /api/ http://127.0.0.1:8001/api/
RequestHeader set X-Forwarded-Proto "https"

ErrorLog ${APACHE_LOG_DIR}/icsc-error.log
CustomLog ${APACHE_LOG_DIR}/icsc-access.log combined
</VirtualHost>
EOF

sudo a2disssite 000-default.conf
sudo a2ensite icsc.conf
sudo apache2ctl configtest
sudo systemctl reload apache2

```

Open `http://school.example.com` — the login page should appear.

8. HTTPS with Let's Encrypt

```

sudo apt install -y certbot python3-certbot-apache
sudo certbot --apache -d school.example.com -m you@example.com --agree-tos --redirect
# Cert auto-renews via certbot.timer

```

After this, the site is served at `https://school.example.com`.

9. Firewall (recommended)

```

sudo ufw allow OpenSSH
sudo ufw allow 'Apache Full'
sudo ufw enable
sudo ufw status

```

10. First-login test

Hit `https://school.example.com`, log in with the admin email & password set in `.env`. The backend auto-seeds the admin + demo data on first startup. Credentials are also written to `/opt/icsc/memory/test_credentials.md`.

Day-to-day operations

Action	Command
View API logs	<code>sudo journalctl -u icsc-backend -f</code>
Restart API	<code>sudo systemctl restart icsc-backend</code>
Rebuild frontend after code changes	<code>cd /opt/icsc/frontend && yarn build</code>
Update backend code	<code>sudo systemctl restart icsc-backend</code>
Mongo backup	<code>mongodump --db icsc_school_db --out /opt/icsc/backups/\$(date +%F)</code>
Add SendGrid key	<code>edit /opt/icsc/backend/.env</code> , then <code>sudo systemctl restart icsc-backend</code>

Common gotchas

- **502 from Apache** — backend not running. `sudo systemctl status icsc-backend` .
- **CORS errors in browser** — `CORS_ORIGINS` / `FRONTEND_URL` in `.env` must exactly match the `https://...` origin.
- **Cookies not persisting** — cookies are `Secure` , only work over HTTPS. Ensure step 8 succeeded.
- **MongoDB connection refused** — `sudo systemctl status mongod` ; ensure `MONGO_URL` matches the actual port (default 27017).
- **Yarn build runs out of memory** — `NODE_OPTIONS=--max-old-space-size=2048 yarn build` . Use a t3.medium or larger.